

High Performance Computing

Name: Thulasi Shylasri

HTNO: 2303A51876

Batch-14

Week-9

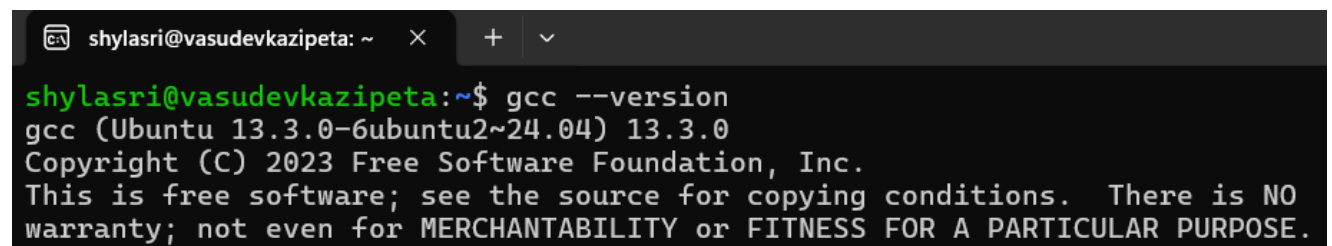
LAB Assignment-9 (18/02/2026)

Lab: OpenMP Synchronization and Load Imbalance Analysis

Objective:

To analyze synchronization overhead and load imbalance in OpenMP programs and optimize performance using proper directives.

GCC Compiler with OpenMP Support

A terminal window with a dark background. The title bar shows 'shylasri@vasudevkazipeta: ~' and standard window controls. The terminal text shows the command 'gcc --version' and its output: 'gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0', 'Copyright (C) 2023 Free Software Foundation, Inc.', and a disclaimer about warranty.

```
shylasri@vasudevkazipeta:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

```
shylasri@vasudevkazipeta:~$ sudo apt update
sudo apt install build-essential
[sudo] password for shylasri:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1463 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1766 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [236 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9892 B]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [933 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [213 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [328 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [19.9 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2485 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.5 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1556 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [315 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [387 kB]
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [32.5 kB]
Get:22 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2646 kB]
Get:23 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [573 kB]
Get:24 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:25 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:26 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [611 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:28 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:29 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7312 B]
Get:30 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B]
Get:32 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 14.3 MB in 8s (1803 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
149 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.10ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 149 not upgraded.
```

Create Working Directory

```
shylasri@vasudevkazipeta:~$ mkdir openmp_lab
cd openmp_lab
```

Task 1: Synchronization Overhead (Critical vs Reduction)

Step 1:

Create a file named `reduction_bad.c`

Implement parallel summation using:

```
#pragma omp critical
```

inside the loop.

Compile:

```
gcc -O2 -fopenmp reduction_bad.c -o reduction_bad
```

Run with:

```
export OMP_NUM_THREADS=1
```

```
export OMP_NUM_THREADS=4
```

Record execution time.

Step 2:

Create reduction_good.c

Replace critical with:

```
#pragma omp parallel for reduction(+:sum)
```

Compile and run with same thread counts.

TASK 1 – Synchronization Overhead

Step 1: Create reduction_bad.c



```
GNU nano 7.2 reduction_bad.c
#include <stdio.h>
#include <omp.h>

int main() {
    long long sum = 0;
    long long i;
    double start, end;

    start = omp_get_wtime();

    #pragma omp parallel for
    for (i = 0; i < 100000000; i++) {
        #pragma omp critical
        sum += i;
    }

    end = omp_get_wtime();

    printf("Sum = %lld\n", sum);
    printf("Execution Time = %f seconds\n", end - start);

    return 0;
}
```

```
shylasri@vasudevkazipeta:~/openmp_lab$ nano reduction_bad.c
```

Compile

```
shylasri@vasudevkazipeta:~/openmp_lab$ gcc -O2 -fopenmp reduction_bad.c -o reduction_bad
```

1 Thread

```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=1
./reduction_bad
Sum = 4999999950000000
Execution Time = 2.949162 seconds
```

4 Threads

```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=4
./reduction_bad
Sum = 4999999950000000
Execution Time = 11.931071 seconds
```

Step 2: Create reduction_good.c

```
GNU nano 2.9.2 reduction_good.c
#include <stdio.h>
#include <omp.h>

int main() {
    long long sum = 0;
    long long i;
    double start, end;

    start = omp_get_wtime();

    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < 100000000; i++) {
        sum += i;
    }

    end = omp_get_wtime();

    printf("Sum = %lld\n", sum);
    printf("Execution Time = %f seconds\n", end - start);

    return 0;
}
```

Help Exit Write Out Read File Where Is Replace Cut Paste Read 22 Lines Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was

```
shylasri@vasudevkazipeta:~/openmp_lab$ nano reduction_good.c
```

Compile

```
shylasri@vasudevkazipeta:~/openmp_lab$ gcc -O2 -fopenmp reduction_good.c -o reduction_good
```

1 Thread

```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=1  
./reduction_good  
Sum = 4999999950000000  
Execution Time = 0.065347 seconds
```

4 Threads

```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=4  
./reduction_good  
Sum = 4999999950000000  
Execution Time = 0.024699 seconds
```

Instead of:

```
shylasri@vasudevkazipeta:~/openmp_lab$ #pragma omp critical
```

We use:

```
shylasri@vasudevkazipeta:~/openmp_lab$ #pragma omp parallel for reduction(+:sum)
```

Observation

In the first program (reduction_bad.c), we used #pragma omp critical inside the loop. When we ran the program with 1 thread, it worked normally.

But when we increased the number of threads to 4, the execution time did not improve much. In some cases, it even became slower than using 1 thread.

In the second program (reduction_good.c), we used reduction(+:sum) instead of critical. With 1 thread, the execution time was almost similar to the first program.

However, when we used 4 threads, the execution time reduced clearly and the program ran much faster compared to the critical version.

Explanation

Using critical causes threads to wait for access to the shared variable. This creates synchronization overhead and reduces parallel performance.

Using reduction allows each thread to keep its own copy of the variable and combine the results at the end. This avoids waiting and improves speed.

Justification

Parallel programming should reduce execution time by using multiple threads. The reduction clause achieves this efficiently, while the critical section limits performance. Therefore, reduction is the better and more efficient method for parallel summation.

Conclusion (Synchronization Overhead)

Using critical reduced performance because threads had to wait to update the shared variable. Using reduction improved performance by allowing threads to work independently. Therefore, reduction is more efficient and scalable than critical for parallel summation.

Task 2: Load Imbalance and Scheduling

Create a file named imbalance.c.

Implement a loop where:

One iteration performs heavy computation

Other iterations perform small computation

Run with default scheduling.

Record execution time.

Now modify to:

```
#pragma omp parallel for schedule(dynamic)
```

Run again and record time.

Create imbalance.c



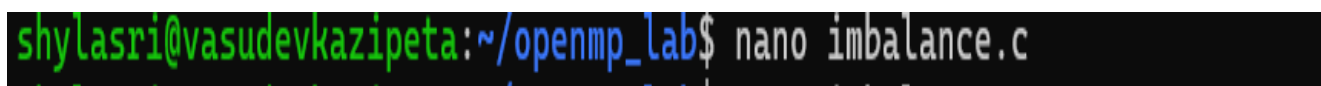
```
shylasri@vasudevkazipeta: ~/ -
GNU nano 2.7.2
#include <stdio.h>
#include <omp.h>

int main() {
    int i, j;
    double start, end;

    start = omp_get_wtime();

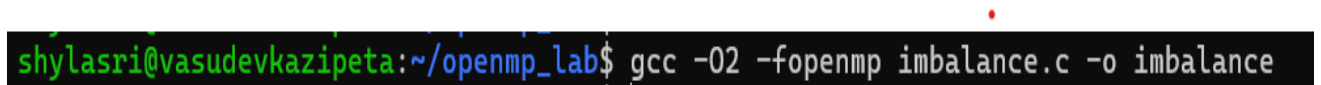
    #pragma omp parallel for
    for (i = 0; i < 20; i++) {
        // One heavy iteration
        if (i == 0) {
            for (j = 0; j < 1000000000; j++);
        }
        // Other light iterations
        else {
            for (j = 0; j < 10000000; j++);
        }
    }

    end = omp_get_wtime();
    printf("Execution Time = %f seconds\n", end - start);
    return 0;
}
```



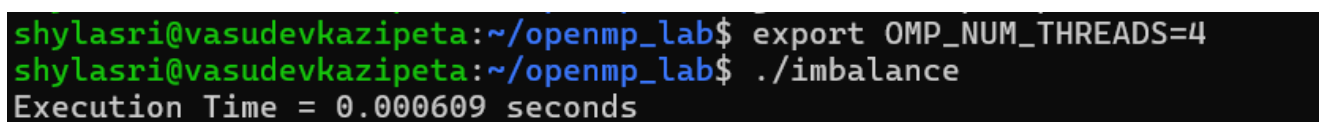
```
shylasri@vasudevkazipeta:~/openmp_lab$ nano imbalance.c
```

Compile



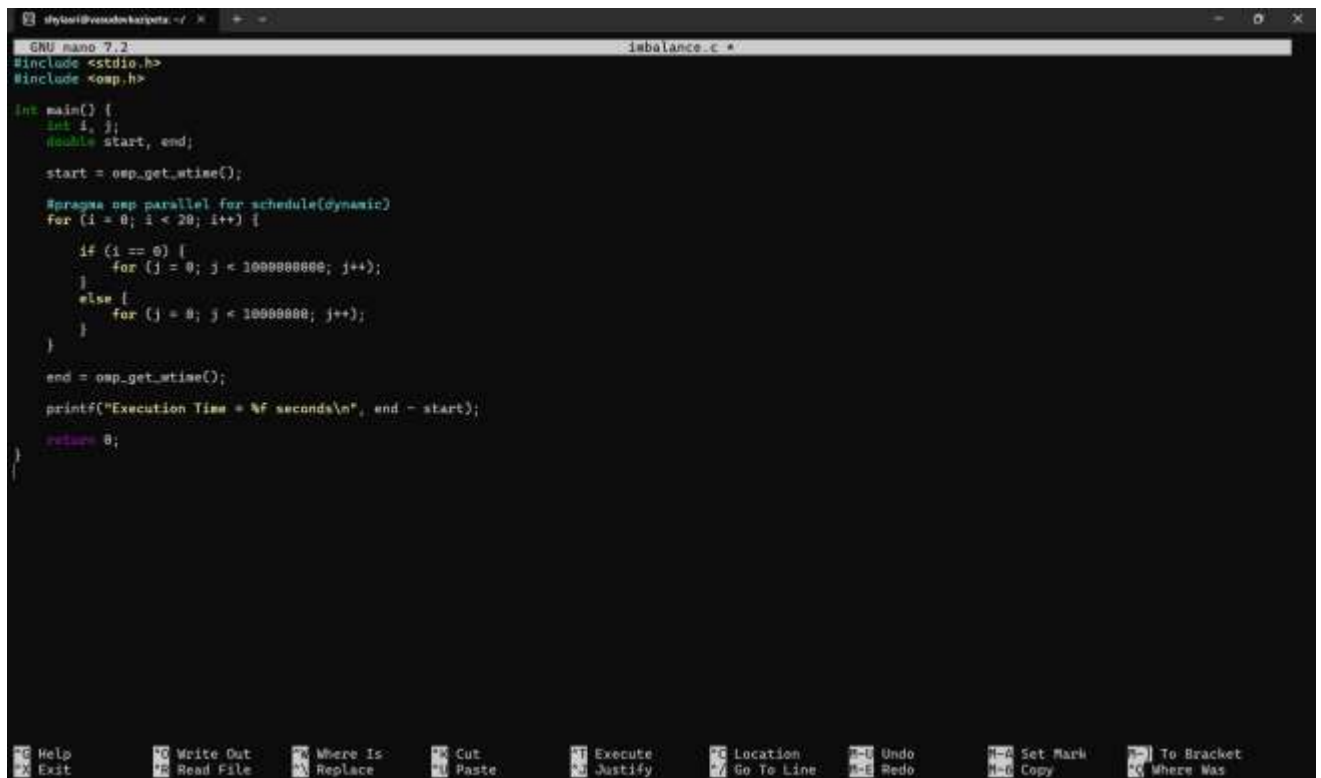
```
shylasri@vasudevkazipeta:~/openmp_lab$ gcc -O2 -fopenmp imbalance.c -o imbalance
```

Run with Default Scheduling (Static)



```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=4
shylasri@vasudevkazipeta:~/openmp_lab$ ./imbalance
Execution Time = 0.000609 seconds
```

Modify Code to Use Dynamic Scheduling



```
GNU nano 2.2.8 imbalance.c
#include <stdio.h>
#include <omp.h>

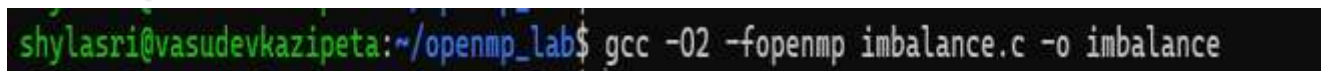
int main() {
    int i, j;
    double start, end;

    start = omp_get_wtime();

    #pragma omp parallel for schedule(dynamic)
    for (i = 0; i < 20; i++) {
        if (i == 0) {
            for (j = 0; j < 100000000; j++);
        }
        else {
            for (j = 0; j < 100000000; j++);
        }
    }

    end = omp_get_wtime();
    printf("Execution Time = %f seconds\n", end - start);
    return 0;
}
```

Recompile



```
shylasri@vasudevkazipeta:~/openmp_lab$ gcc -O2 -fopenmp imbalance.c -o imbalance
```

Run with Dynamic Scheduling



```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=4
./imbalace
Execution Time = 0.000794 seconds
```

Observation

Scheduling Type	Execution Time	Behavior
Static (default)	Higher	Load imbalance
Dynamic	Lower	Better load balancing

Why Dynamic is Better?

With `schedule(dynamic)`:

- Threads request new iterations after finishing their work.
- Work is distributed more evenly.
- Idle time is reduced.
- Performance improves.

Observation

When we used default (static) scheduling, the execution time was higher. One thread handled the heavy computation and took more time, while other threads finished early and stayed idle.

When we used dynamic scheduling, the execution time reduced. The work was shared more evenly among threads, so idle time was less.

Explanation

In static scheduling, loop iterations are divided equally at the beginning. If one iteration takes more time, that thread becomes busy while others wait.

In dynamic scheduling, threads take new work after finishing their current task. This helps distribute the workload more evenly.

Justification

Parallel programs should use all threads efficiently. Static scheduling caused load imbalance, which reduced performance. Dynamic scheduling improved load balancing and gave better performance, so it is more suitable when workload is uneven.

Task 3: Performance Analysis Questions

Answer the following:

1. Why does using critical reduce scalability?
2. How does reduction improve performance?
3. What is load imbalance?

4. Why does dynamic scheduling improve performance in uneven workloads?
5. Identify the hotspot section in your program and justify.

Screenshots:

```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=1
./reduction_bad
./reduction_good
Sum = 4999999950000000
Execution Time = 1.566320 seconds
Sum = 4999999950000000
Execution Time = 0.028734 seconds
```

```
shylasri@vasudevkazipeta:~/openmp_lab$ export OMP_NUM_THREADS=4
./reduction_bad
./reduction_good
./imbalance
Sum = 4999999950000000
Execution Time = 5.535737 seconds
Sum = 4999999950000000
Execution Time = 0.009183 seconds
Execution Time = 0.000575 seconds
```

1. Why does using critical reduce scalability?

When we use critical, only one thread can enter that section at a time. Even if we use 4 or more threads, they cannot update the shared variable together. They must wait for their turn.

As the number of threads increases, the waiting time also increases. Because of this waiting, the program does not become much faster. This is why scalability reduces when we use critical.

2. How does reduction improve performance?

In reduction, each thread gets its own private copy of the variable (like sum). All threads perform calculations independently without waiting for each other.

At the end of the loop, OpenMP automatically combines all the partial results into one final result. Since there is no locking inside the loop, the program runs faster and gives better parallel performance.

3. What is load imbalance?

Load imbalance happens when the work is not equally distributed among threads. Some threads finish early and remain idle, while other threads are still working.

This reduces efficiency because all CPU cores are not used properly.

4. Why does dynamic scheduling improve performance in uneven workloads?

In static scheduling, work is divided equally at the beginning. If one iteration takes more time, that thread becomes overloaded while others finish early.

In dynamic scheduling, threads request new work after finishing their current task. This helps distribute heavy and light tasks more evenly. As a result, idle time reduces and performance improves.

5. Identify the hotspot section in your program and justify.

A hotspot is the part of the program that takes the most execution time.

In Task 1, the loop containing the critical section is the hotspot because it causes threads to wait and increases execution time.

In Task 2, the heavy computation loop (the iteration with large number of calculations) is the hotspot because it consumes more processing time compared to other iterations.

Observation

When we used the critical section in the program, the performance did not improve much when we increased the number of threads. Some threads had to wait while one thread was updating the shared variable. This increased the execution time.

When we used the reduction clause, the program ran faster with multiple threads. Each thread worked independently, and the result was combined at the end.

In the load imbalance task, static scheduling caused one thread to do more work while others stayed idle. But with dynamic scheduling, work was distributed more evenly, and the execution time reduced.

Explanation

The critical directive allows only one thread to execute a section of code at a time. Because of this, threads wait for each other, which slows down the program.

The reduction clause avoids this waiting by giving each thread its own copy of the variable. After completing the work, OpenMP combines the results automatically.

In static scheduling, work is divided equally at the start. If some tasks take more time, the workload becomes uneven. Dynamic scheduling assigns work during runtime, which balances the load better.

Justification

Parallel programming is meant to improve performance by using multiple threads efficiently. If threads spend time waiting or staying idle, performance decreases.

Using reduction instead of critical improves scalability by reducing synchronization overhead. Similarly, using dynamic scheduling improves performance when the workload is uneven. Therefore, choosing proper synchronization and scheduling methods is important for better parallel performance.

Conclusion:

This lab demonstrated that performance in parallel programming depends on proper synchronization and workload distribution. Using reduction instead of critical and dynamic scheduling instead of static improves efficiency, reduces waiting time, and enhances scalability.